

CYMANABU

Cybersecurity Internship Program

WEEK 2 TASK SUBMISSION

Internship Domain

Cybersecurity – Network Security & Traffic Analysis

Tools Used

Kali Linux | Wireshark | VMware

Submitted By: Saif Sami

College: Oriental College
of Technology

Branch: Cybersecurity

Internship: Cybersecurity Internship

TABLE OF CONTENTS

1. Introduction.....	1
2. Abstract.....	2
3. Project Objectives	3
4. Lab Setup & Architecture	4
5. Tools Used	6
6. Methodology	7
7. DNS Traffic Analysis	8
8. TCP Traffic Analysis	10
9. UDP Traffic Analysis	13
10. ICMP Traffic Analysis	15
11. HTTP Traffic Analysis	18
12. Findings and Observations	21
13. Security Analysis & Risk Identification	22
14. MITRE ATT&CK Mapping	23
15. Remediation & Security Recommendations	24
16. Conclusion & Key Learnings	25
17. References & Evidence	26

1. Introduction

During Week 2 of the Cybersecurity Internship, practical work was performed on **Network Traffic Analysis using Wireshark**. The objective of this week was to understand how network packets are generated, captured, filtered, and analyzed in a controlled cybersecurity laboratory environment.

A Kali Linux virtual machine was configured in VMware and Wireshark was used as the primary packet-analysis tool. Different types of network traffic, including **DNS, TCP, UDP, ICMP, and HTTP**, were generated and captured for analysis.

The captured packets were examined to identify source and destination IP addresses, port numbers, protocols, TCP flags, packet types, and transmitted data. Screenshots and observations were documented as evidence of the practical work.

This activity provided practical exposure to network monitoring, packet analysis, protocol behavior, and basic network-security assessment techniques.

ABSTRACT

Network traffic analysis is an important process in cybersecurity that helps understand and monitor communication between devices over a network. This project focuses on capturing and analyzing network packets using Wireshark in a Kali Linux virtual machine.

The practical work involved analyzing DNS, TCP, UDP, ICMP, and HTTP traffic. Wireshark was used to examine packet details such as source and destination IP addresses, port numbers, protocol types, TCP handshake packets, and HTTP data.

The project provided practical experience in packet capture, protocol analysis, and understanding network communication. It also demonstrated how unencrypted HTTP traffic can expose readable webpage content during transmission.

PROJECT OBJECTIVES

The main objectives of this project are:

1. To understand the fundamentals of network traffic analysis.
2. To capture live network packets using Wireshark.
3. To analyze DNS, TCP, UDP, ICMP, and HTTP protocols.
4. To identify source and destination IP addresses and port numbers.
5. To observe the TCP 3-way handshake.
6. To examine DNS query and response communication.
7. To understand the risks associated with unencrypted HTTP traffic.
8. To develop practical skills in packet capture and network troubleshooting.

LAB SETUP & ARCHITECTURE.

6. LAB SETUP & ARCHITECTURE

6.1 Lab Environment

The project was performed in a Kali Linux virtual machine using Wireshark for network traffic capture and analysis.

Component	Description
Operating System	Kali Linux
Virtualization	VMware
Network Interface	eth0
Packet Analysis Tool	Wireshark
Test Environment	Personal virtual machine and network
Traffic Types	DNS, TCP, UDP, ICMP, HTTP

6.2 Network Architecture

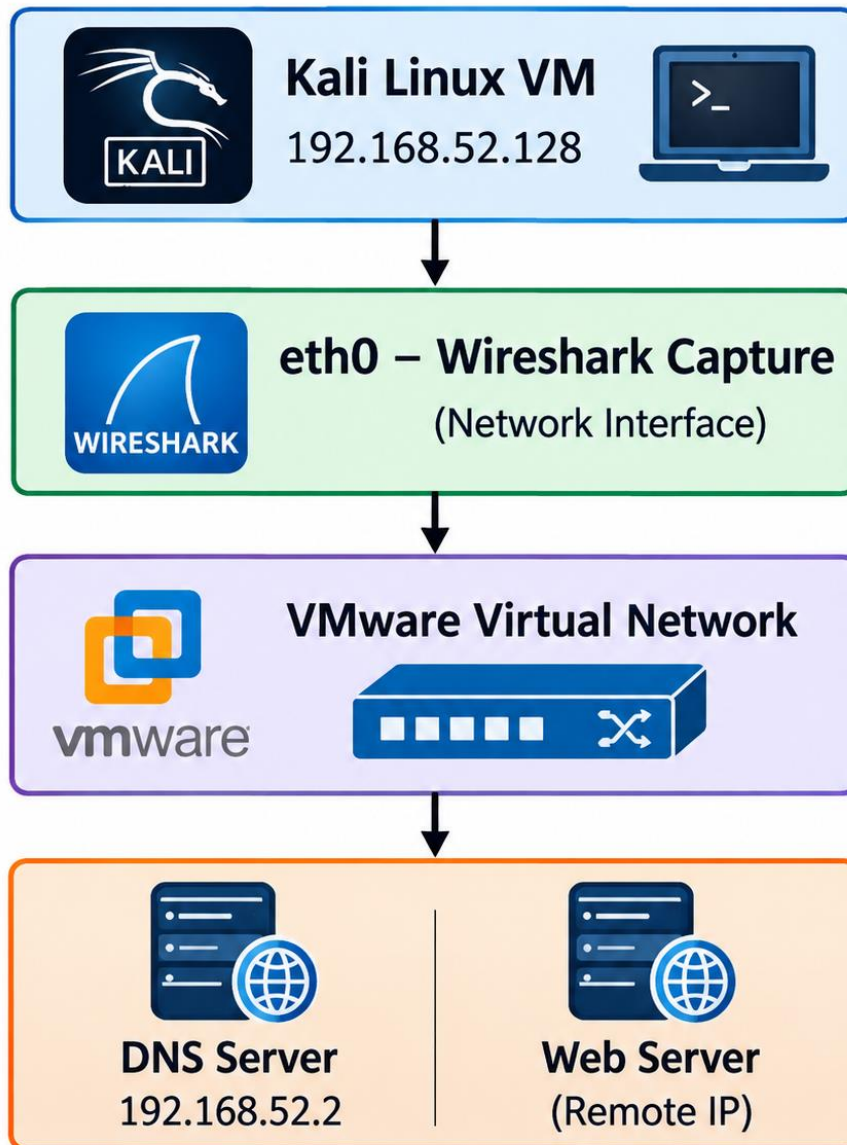
The Kali Linux VM generated network traffic and captured packets through its eth0 interface. The traffic included communication with the configured DNS server and a remote web server.

Network Flow:

Kali Linux VM (192.168.52.128)
↓
eth0 Network Interface
↓
VMware Virtual Network
↓
DNS Server (192.168.52.2) / Remote Web Server

6.3 Lab Safety

All packet captures were performed in the user's own virtual machine and for educational analysis. The project focused on observing network traffic and protocol behavior.



Kali Linux (eth0) captures traffic from the VMware virtual network, which communicates with DNS / Web Server (192.168.52.2 / Remote IP).

4. Tools Used

The following tools and technologies were used for performing the network traffic analysis:

Tool / Technology Purpose

Kali Linux	Operating system used for the security lab
VMware	Used to run the Kali Linux virtual machine
Wireshark	Capturing and analyzing network packets
Terminal	Generating network traffic and running commands
TCP/IP	Network communication protocols analyzed
DNS	Domain name resolution traffic analysis
HTTP	Unencrypted web traffic analysis
ICMP	Connectivity and diagnostic traffic analysis
UDP	Connectionless network traffic analysis
TCP	Reliable connection-oriented traffic analysis

4.1 Traffic Generation Commands

The following commands were used to generate traffic for analysis:

- `nslookup example.com` — DNS traffic
- `curl http://example.com` — TCP/HTTP traffic
- `ping -c 4 192.168.52.2` — ICMP traffic
- `curl http://neverssl.com` — HTTP traffic

5. Methodology

The network traffic analysis was performed using a controlled Kali Linux virtual lab. Wireshark was used to capture packets generated by different network activities.

5.1 Step-by-Step Methodology

1. Started the Kali Linux virtual machine in VMware.
2. Verified the network interface eth0 and IP address 192.168.52.128.
3. Opened Wireshark and selected the eth0 interface for packet capture.
4. Generated DNS traffic using nslookup example.com.
5. Generated TCP and HTTP traffic using curl.
6. Generated ICMP traffic using ping.
7. Applied Wireshark display filters to identify specific protocols.
8. Analyzed source and destination IP addresses, ports, packet types, and communication patterns.
9. Captured screenshots of important packets as evidence.
10. Recorded the observations and findings for each analyzed protocol.

Used5.2 Wireshark Filters

Protocol	Wireshark Filter
----------	------------------

DNS	Dns
-----	-----

TCP	Tcp
-----	-----

UDP	Udp
-----	-----

ICMP	Icmp
------	------

HTTP	http
------	------

Specific IP ip.addr == 192.168.52.128

5.3 Analysis Process

The captured packets were examined individually to identify communication between the source and destination systems. Protocol-specific fields such as IP addresses, port numbers, packet types, TCP flags, and packet contents were inspected where applicable.

6. DNS Traffic Analysis

6.1 Objective

The objective of DNS traffic analysis was to observe how a domain name is resolved into an IP address and to identify the communication between the Kali Linux system and the DNS server.

6.2 Traffic Generation

The following command was used to generate DNS traffic:

```
nslookup example.com
```

6.3 Wireshark Filter

```
dns
```

6.4 Packet Details

Parameter	Observation
Source IP	192.168.52.128
Destination IP	192.168.52.2
Protocol	DNS
Destination Port	53
Query	example.com
DNS Server	192.168.52.2

The DNS query was sent from the Kali Linux system to the DNS server. The DNS server returned the resolved IP addresses for example.com.

6.5 Resolved Addresses

The observed DNS response included the following addresses:

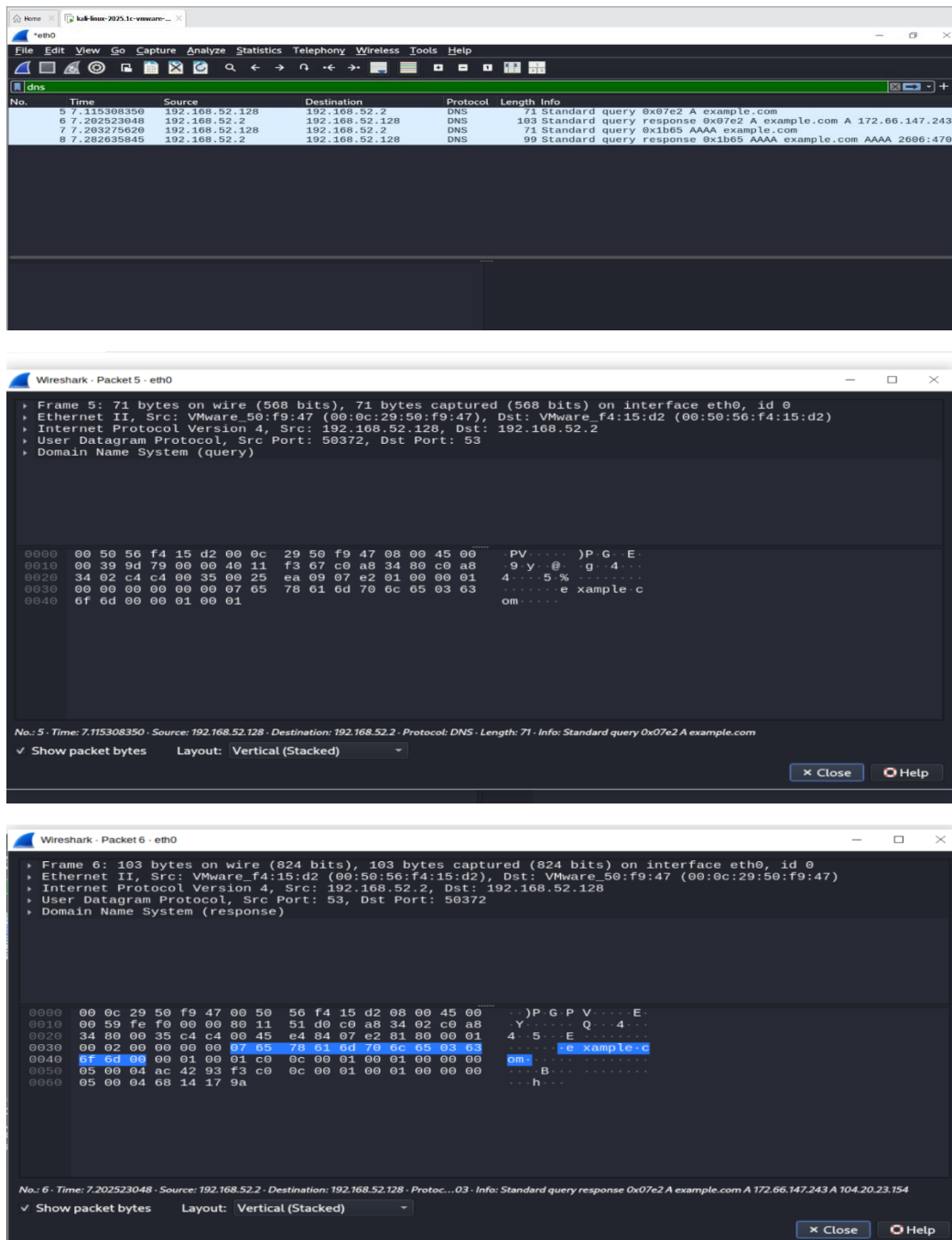
- 104.20.23.154
- 172.66.147.243
- 2606:4700:90d5:72db:f2c5:8af:ef6b:ff98

6.6 Observation

Wireshark showed a DNS query from 192.168.52.128 to 192.168.52.2 and a corresponding DNS response from the DNS server. The traffic demonstrated the basic domain-name resolution process.

6.7 Evidence

Figure 2: DNS Traffic Capture



Caption: Figure 2: DNS Query and Response Captured Using Wireshark

7. TCP Traffic Analysis

7.1 Objective

The objective of TCP traffic analysis was to observe the TCP three-way handshake and understand how a reliable connection is established between two systems.

7.2 Traffic Generation

The following command was used to generate TCP traffic:

```
curl http://example.com
```

7.3 Wireshark Filter

```
tcp
```

7.4 TCP Three-Way Handshake

The captured traffic showed the TCP connection establishment process:

1. **SYN** — 192.168.52.128 → 104.20.23.154
2. **SYN-ACK** — 104.20.23.154 → 192.168.52.128
3. **ACK** — 192.168.52.128 → 104.20.23.154

This sequence represents the TCP three-way handshake used to establish a connection before data transmission.

7.5 Packet Analysis

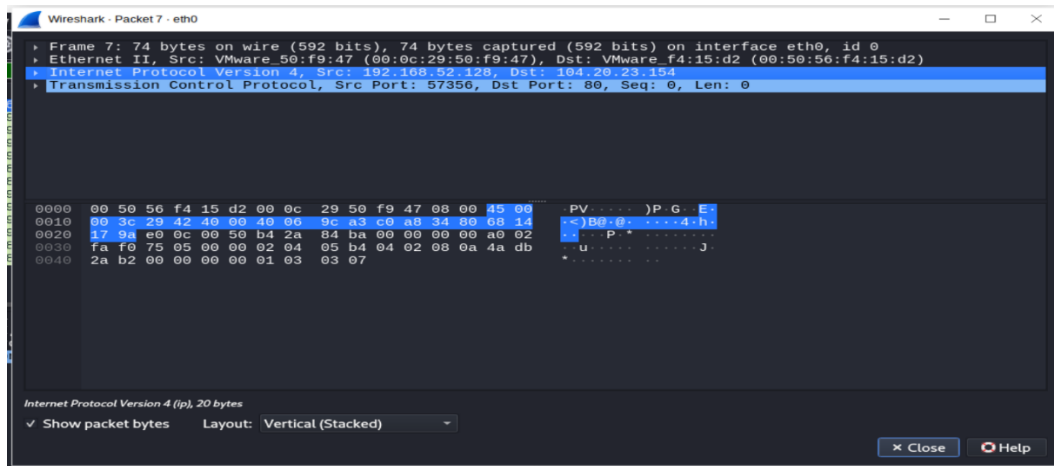
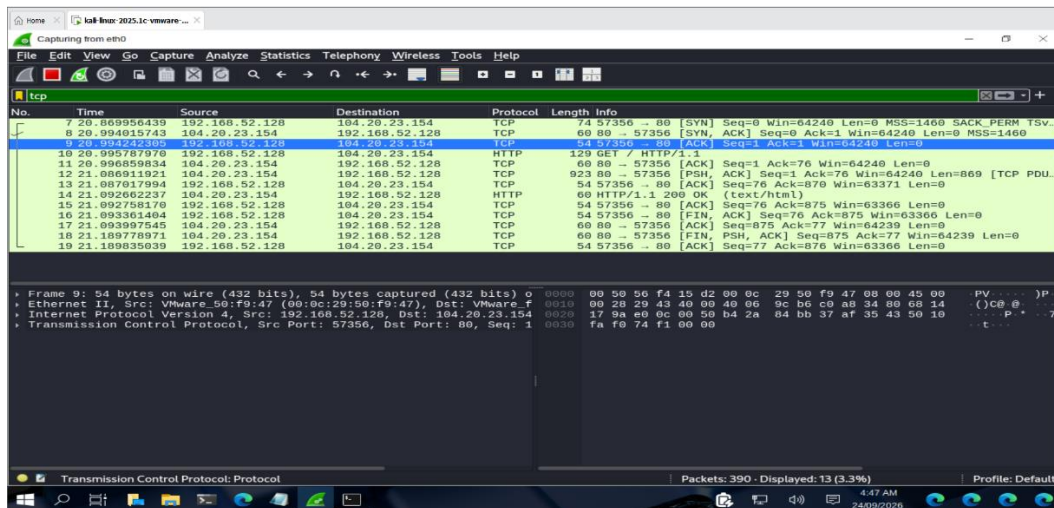
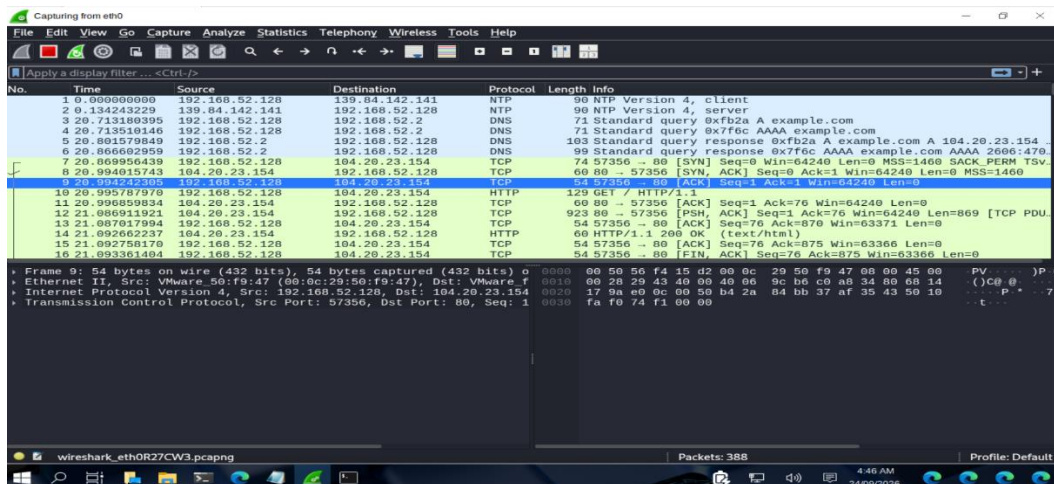
Parameter	Observation
Source IP	192.168.52.128
Destination IP	104.20.23.154
Protocol	TCP
Connection Process	SYN → SYN-ACK → ACK
Purpose	Connection establishment

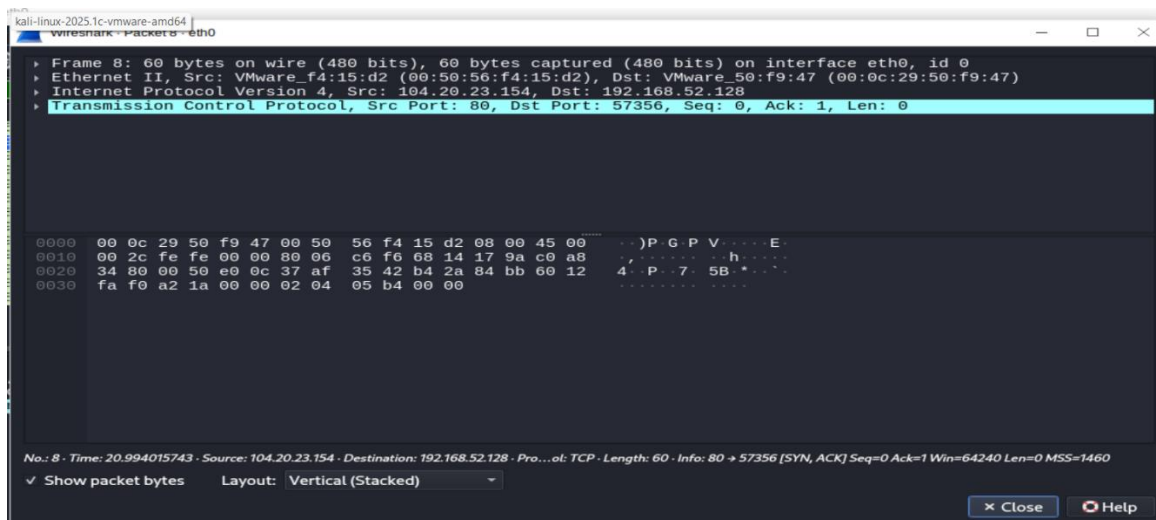
7.6 Observation

The Wireshark capture clearly showed the TCP three-way handshake. The client first sent a SYN packet, the remote server responded with SYN-ACK, and the client completed the process with an ACK packet.

7.7 Evidence

Figure 3: TCP Three-Way Handshake





Caption: *Figure 3: TCP Three-Way Handshake Captured Using Wireshark*

8. UDP Traffic Analysis

8.1 Objective

The objective of UDP traffic analysis was to observe connectionless communication and identify the UDP packet used for DNS communication.

8.2 Traffic Generation

The following command was used to generate UDP-based DNS traffic:

```
nslookup example.com
```

8.3 Wireshark Filter

```
udp
```

8.4 Packet Details

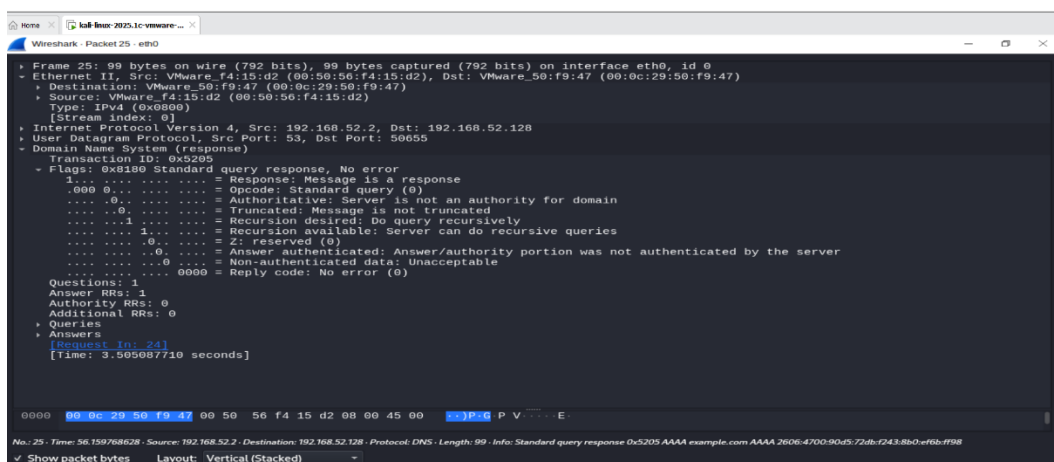
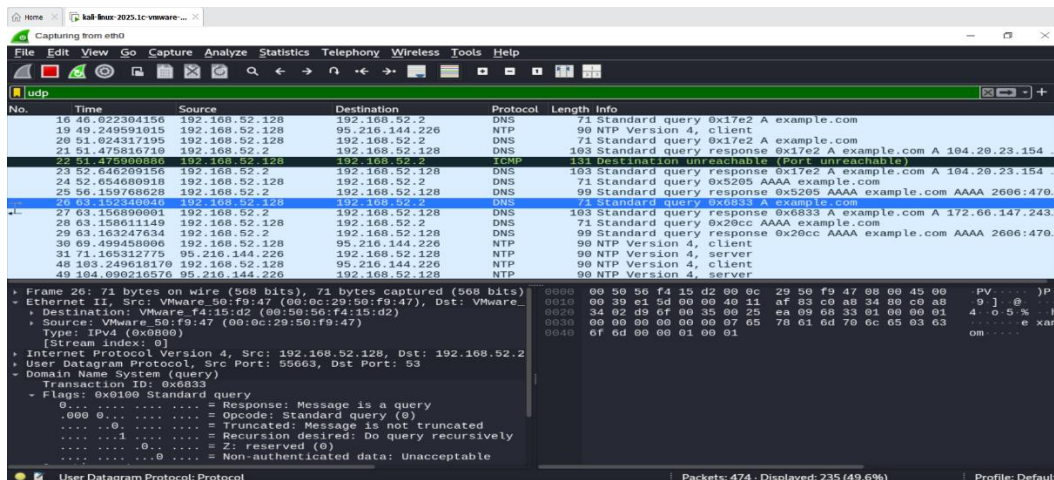
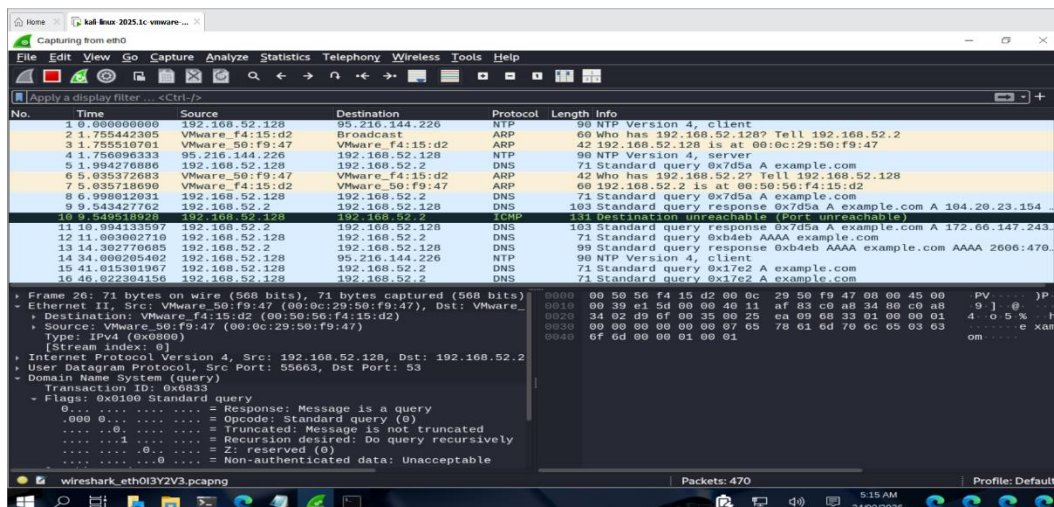
Parameter	Observation
Frame	26
Source IP	192.168.52.128
Destination IP	192.168.52.2
Protocol	UDP
Source Port	55663
Destination Port	53
Packet Length	71 bytes

8.5 Observation

The captured packet showed UDP communication from the Kali Linux system to the DNS server. The destination port was **53**, which is commonly used for DNS queries. Unlike TCP, UDP does not establish a connection using a three-way handshake before transmitting data.

8.6 Evidence

Figure 4: UDP Traffic Capture



Caption: Figure 4: UDP DNS Traffic Captured Using Wireshark

9. ICMP Traffic Analysis

9.1 Objective

The objective of ICMP traffic analysis was to observe ICMP Echo Request and Echo Reply packets used to test network connectivity.

9.2 Traffic Generation

The following command was used:

```
ping -c 4 192.168.52.2
```

9.3 Wireshark Filter

icmp

9.4 Packet Details

Parameter	Observation
Request Source	192.168.52.128
Request Destination	192.168.52.2
Reply Source	192.168.52.2
Reply Destination	192.168.52.128
Protocol	ICMP
Request	Echo Request
Response	Echo Reply
Packet Size	98 bytes

9.5 ICMP Communication

The captured traffic showed:

Echo Request:

192.168.52.128 → 192.168.52.2

Echo Reply:

192.168.52.2 → 192.168.52.128

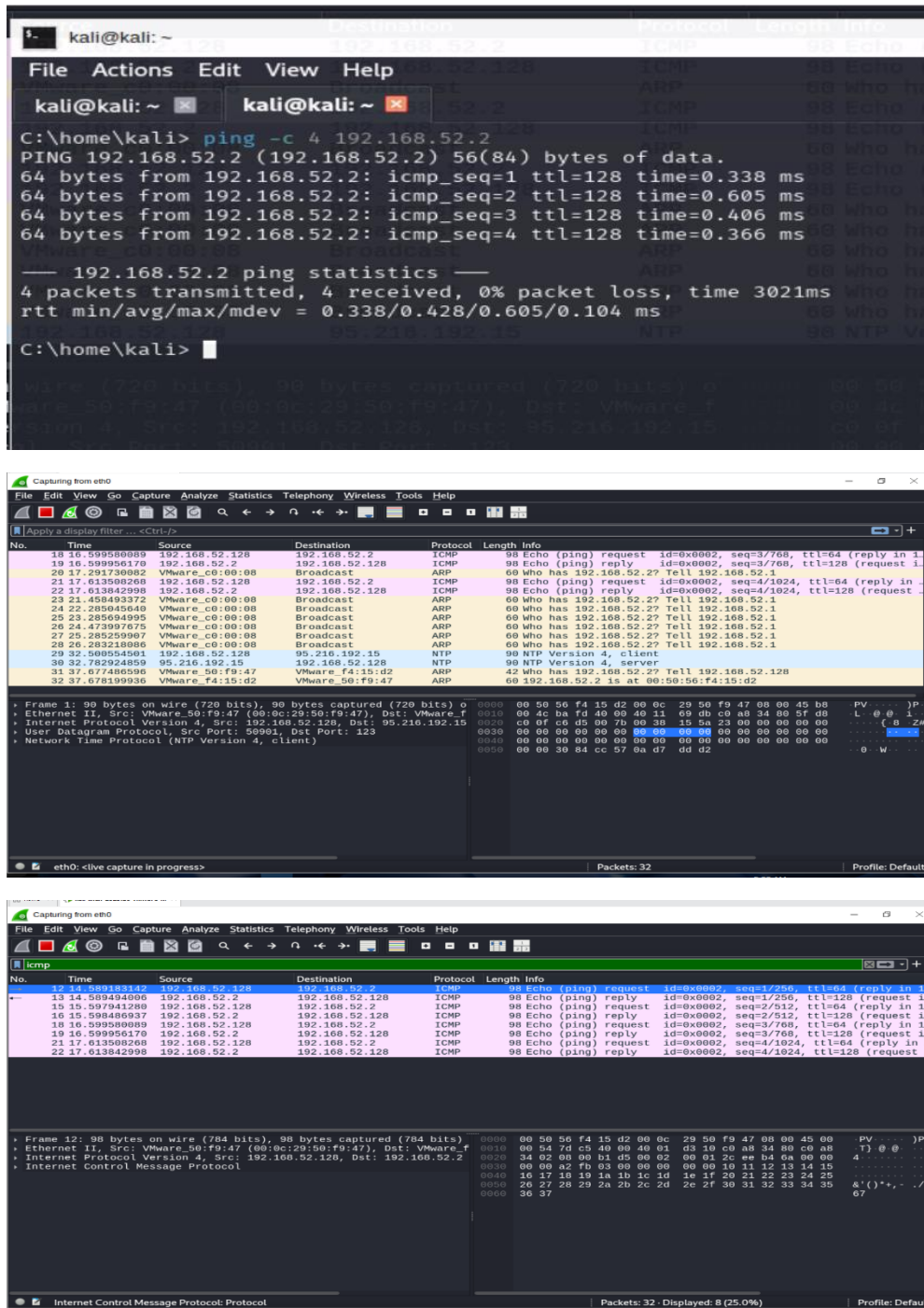
The Echo Request is sent to check connectivity, while the Echo Reply indicates that the destination responded.

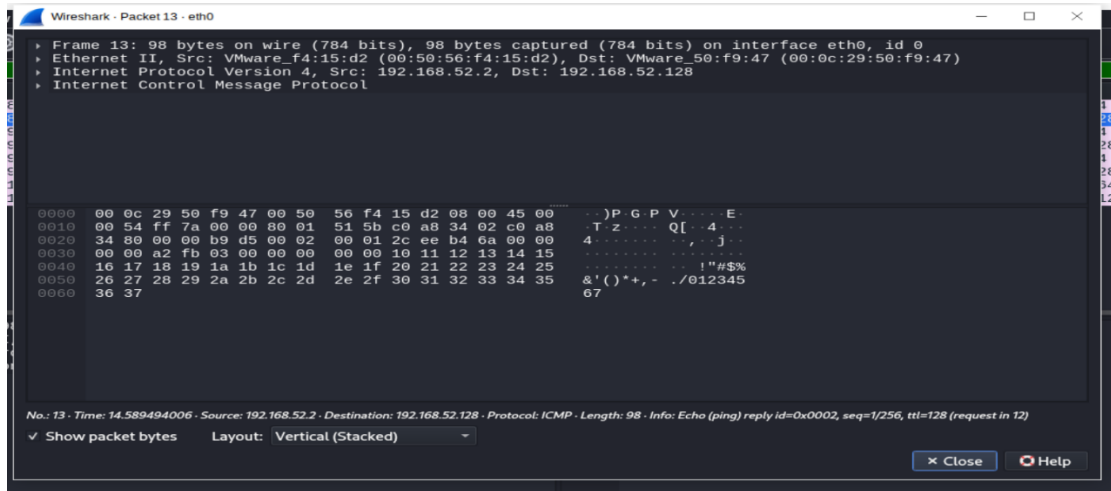
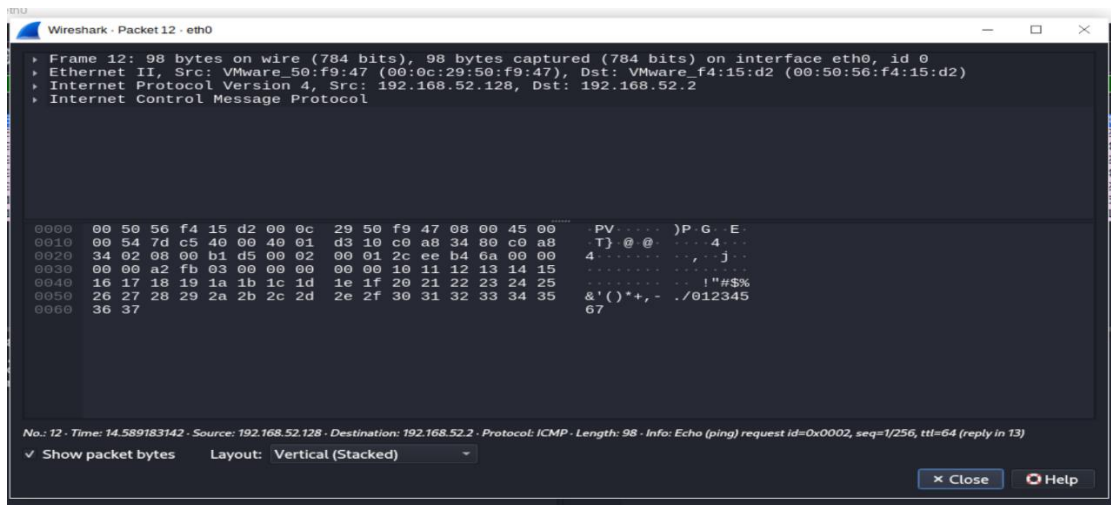
9.6 Observation

Wireshark successfully captured ICMP Echo Request and Echo Reply packets between the Kali Linux system and the DNS server. This demonstrated basic network connectivity between the two systems.

9.7 Evidence

Figure 5: ICMP Traffic Capture





Caption: *Figure 5: ICMP Echo Request and Echo Reply Captured Using Wireshark*

10. HTTP Traffic Analysis

10.1 Objective

The objective of HTTP traffic analysis was to observe unencrypted web traffic and examine the data transmitted between the client and web server.

10.2 Traffic Generation

The following command was used:

```
curl http://neverssl.com
```

10.3 Wireshark Filter

http

10.4 Packet Details

Parameter	Observation
Source IP	34.223.124.45
Destination IP	192.168.52.128
Protocol	HTTP over TCP
Source Port	80
Destination Port	36820
Frame	68
TCP Payload	1661 bytes
Frame Size	1715 bytes
Content Type	text/html

10.5 Traffic Analysis

The captured packet contained HTTP response data from the web server. The reassembled TCP data contained readable HTML content from the website.

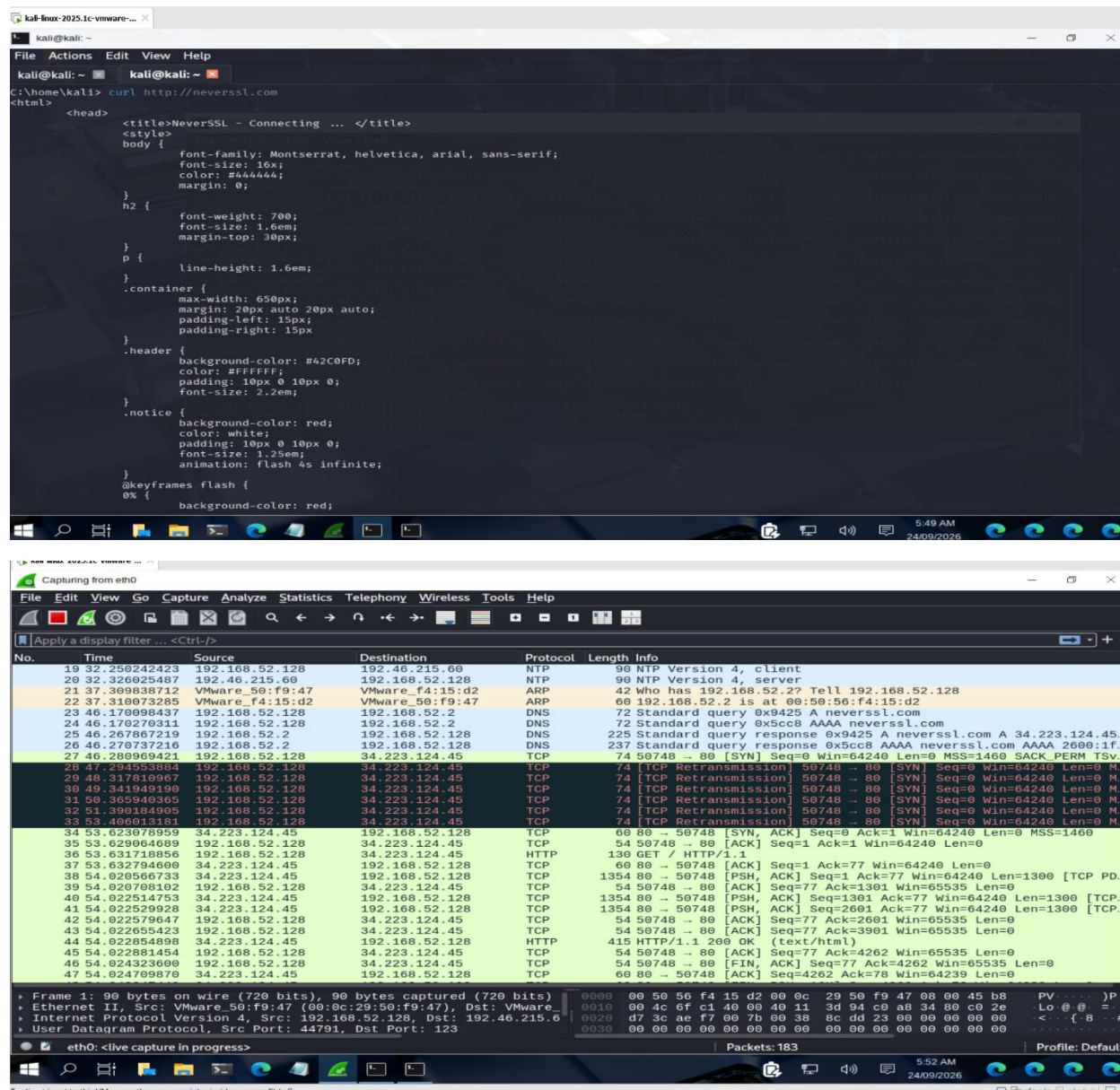
Because HTTP traffic is not encrypted, the transmitted HTML content could be inspected directly in the packet capture.

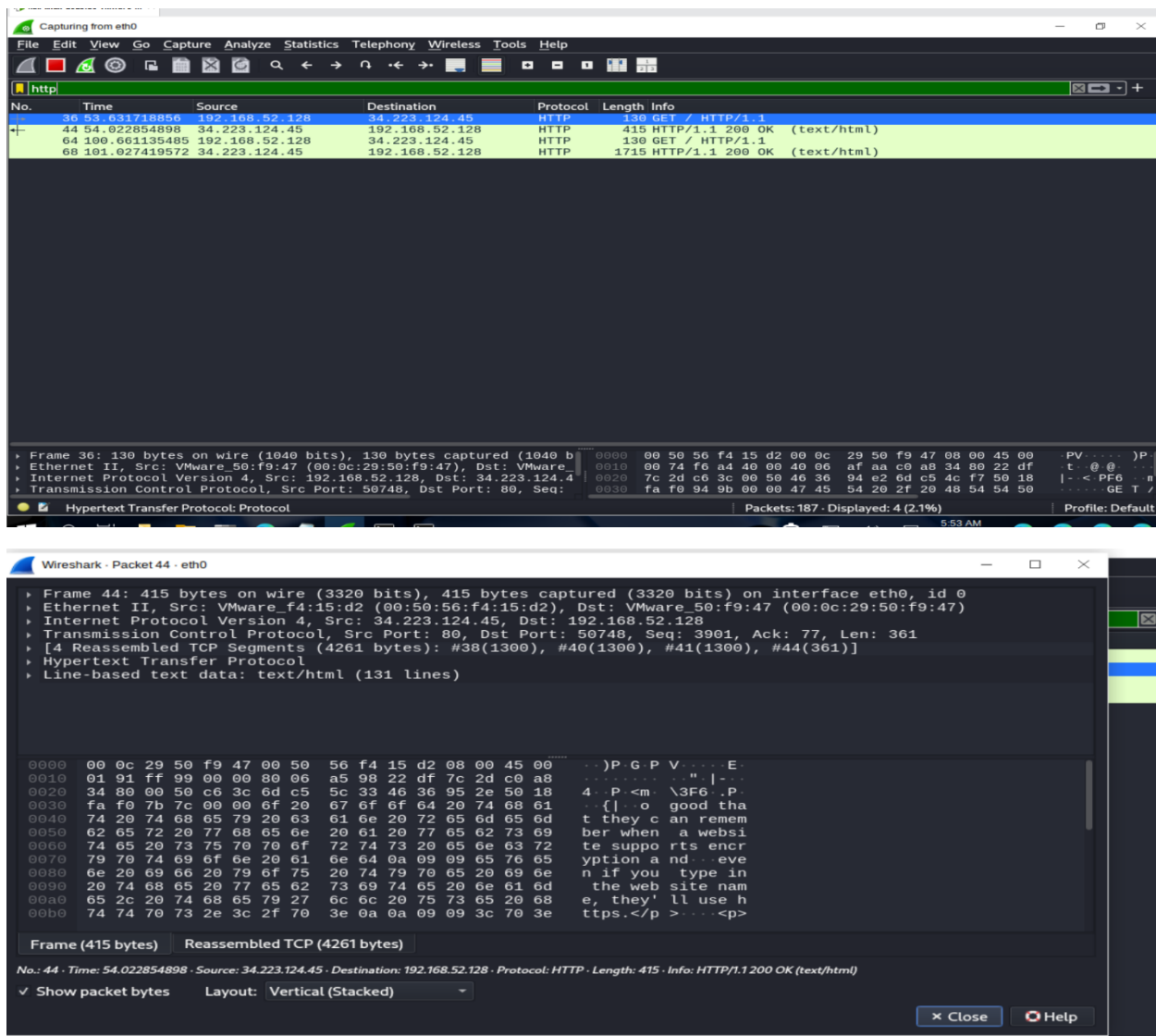
10.6 Observation

The Wireshark capture demonstrated that HTTP data can be viewed in readable form when transmitted without encryption. This highlights the security advantage of using HTTPS for web communication.

10.7 Evidence

Figure 6: HTTP Traffic Capture





Caption: Figure 6: HTTP Traffic and Readable HTML Data Captured Using Wireshark

11. Findings and Observations

The captured network traffic was analyzed using Wireshark. The major findings for each protocol are summarized below.

11.1 Summary of Findings

Protocol Key Finding

DNS	DNS query and response traffic was observed between the Kali Linux system and DNS server.
TCP	TCP three-way handshake (SYN, SYN-ACK, ACK) was successfully identified.
UDP	UDP traffic was observed during DNS communication using destination port 53.
ICMP	Echo Request and Echo Reply packets were captured successfully.
HTTP	Readable HTML data was observed in unencrypted HTTP traffic.

11.2 Detailed Observations

DNS:

The DNS capture demonstrated the process of resolving a domain name into IP addresses. Communication was observed between 192.168.52.128 and the DNS server 192.168.52.2.

TCP:

The TCP capture clearly demonstrated the three-way handshake required to establish a reliable connection. The sequence was SYN, SYN-ACK, and ACK.

UDP:

UDP traffic was identified during DNS communication. The captured packet used destination port 53 and demonstrated connectionless communication.

ICMP:

ICMP Echo Request and Echo Reply packets confirmed communication between the Kali Linux system and the DNS server.

HTTP:

The HTTP capture contained readable HTML content. This demonstrated that data transmitted through unencrypted HTTP can be inspected in a packet capture.

11.3 Security Observations

The analysis demonstrated that different network protocols have different communication characteristics. Packet analysis can provide useful information such as source and destination addresses, ports, protocol types, TCP flags, and transmitted content.

The HTTP capture particularly demonstrated the security limitation of transmitting web content without encryption. HTTPS should be used where confidentiality and integrity of web communication are required.

12. Security Analysis & Risk Identification

The captured traffic was analyzed from a network-security perspective to identify observable security considerations.

12.1 Security Analysis

Area	Observation	Security Consideration
DNS	DNS queries and responses were visible	DNS traffic can reveal domain-resolution activity
TCP	Connection establishment was visible	TCP metadata can reveal communicating endpoints and ports
UDP	DNS communication used UDP port 53	UDP does not provide connection establishment like TCP
ICMP	Echo Request and Reply were visible	ICMP can be useful for network discovery and troubleshooting
HTTP	HTML content was readable	Unencrypted HTTP can expose transmitted web data

12.2 Identified Risks

1. Unencrypted HTTP Traffic

The HTTP capture contained readable HTML data. Unencrypted HTTP can expose information transmitted between a client and server.

2. Network Information Disclosure

Packet captures can reveal IP addresses, ports, protocols, and communication patterns. This information may be useful for network troubleshooting but can also provide useful information to an unauthorized observer.

3. DNS Visibility

DNS queries can reveal the domains being resolved by a system. This can provide information about network activity.

12.3 Recommended Security Measures

- Use **HTTPS** instead of unencrypted HTTP.
- Use secure DNS mechanisms where appropriate.

- Apply network segmentation and access controls.
- Monitor unusual network traffic using security monitoring tools.
- Restrict packet-capture access to authorized personnel.
- Use firewalls and appropriate network security controls.
- Regularly analyze network traffic for suspicious activity.

13. MITRE ATT&CK Mapping

MITRE ATT&CK is a knowledge base used to describe adversary tactics and techniques. The network traffic analysis performed in this project is relevant to the analysis of network communications and packet data.

13.1 Relevant Technique

MITRE ATT&CK Technique	Technique ID	Relevance to Project
Network Sniffing	T1040	Wireshark was used to capture and inspect network packets and communication data.

13.2 Network Sniffing – T1040

Network Sniffing refers to monitoring or capturing network traffic to obtain information from network communications.

In this project, Wireshark was used in a controlled laboratory environment to capture DNS, TCP, UDP, ICMP, and HTTP traffic. The captured packets were analyzed to identify communication endpoints, ports, protocols, and transmitted data.

13.3 Project Relevance

The practical exercise demonstrates how packet-capture and traffic-analysis techniques can be used during network security monitoring and investigation. The activity was performed only on the authorized virtual laboratory environment.

13.4 Security Perspective

Network traffic analysis can help security professionals identify unusual communication patterns, investigate network events, and understand how systems communicate across a network.

14. Remediation & Security Recommendations

Based on the network traffic analysis, the following security recommendations can be considered to improve network communication security.

14.1 Use HTTPS

HTTP traffic observed during the analysis contained readable HTML data. Web applications should use HTTPS to encrypt communication between clients and servers.

14.2 Secure DNS Communication

DNS traffic should be protected where appropriate using secure DNS mechanisms such as DNS over HTTPS (DoH) or DNS over TLS (DoT), depending on the network requirements.

14.3 Network Monitoring

Network traffic should be continuously monitored to identify unusual connections, unexpected protocols, suspicious destinations, and abnormal communication patterns.

14.4 Firewall Configuration

Firewalls should be configured according to the principle of least privilege. Unnecessary ports and services should be restricted.

14.5 Access Control

Access to packet-capture tools and network monitoring systems should be limited to authorized users.

14.6 Network Segmentation

Critical systems should be separated using appropriate network segmentation. This can help limit the impact of a security incident.

14.7 Regular Security Analysis

Regular packet and network-log analysis can help identify unusual activity and support incident investigation.

14.8 Summary

The analysis demonstrated the importance of secure communication protocols, network monitoring, access control, and appropriate security controls in protecting network environments.

15. Conclusion & Key Learnings

15.1 Conclusion

The Network Traffic Analysis project provided practical experience in capturing and analyzing network packets using Wireshark in a controlled Kali Linux virtual environment.

DNS, TCP, UDP, ICMP, and HTTP traffic were successfully generated, captured, and analyzed. The analysis helped demonstrate DNS resolution, the TCP three-way handshake, UDP-based communication, ICMP connectivity testing, and the transmission of readable data through HTTP.

The project also demonstrated how network traffic analysis can support security monitoring and investigation. The practical observations highlighted the importance of encrypted communication, network monitoring, access control, and secure network configuration.

15.2 Key Learnings

The major skills and concepts learned during the project include:

- Understanding basic network communication.
- Using Wireshark for packet capture and analysis.
- Applying Wireshark display filters.
- Identifying DNS queries and responses.
- Understanding the TCP three-way handshake.
- Analyzing UDP communication.
- Identifying ICMP Echo Request and Echo Reply packets.
- Analyzing HTTP traffic and readable HTML data.
- Examining source and destination IP addresses.
- Identifying network ports and protocols.
- Understanding basic network security risks.
- Mapping relevant network analysis activity to MITRE ATT&CK.
- Documenting technical findings with screenshots and evidence.

16. References & Evidence

16.1 References

1. Wireshark Documentation — Official documentation for packet capture and network protocol analysis.
2. Kali Linux Documentation — Official documentation for the Kali Linux operating system and security tools.
3. MITRE ATT&CK — Network Sniffing (T1040) technique reference.
4. RFC 768 — User Datagram Protocol (UDP).
5. RFC 793 — Transmission Control Protocol (TCP).
6. RFC 792 — Internet Control Message Protocol (ICMP).

16.2 Project Evidence

The following screenshots were collected during the practical implementation:

No.	Evidence	Purpose
1	NS-001_DNS_Traffic.png	DNS query and response
2	NS-001_TCP_Handshake.png	TCP three-way handshake
3	NS-001_UDP_Traffic.png	UDP/DNS communication
4	NS-001_ICMP_Traffic.png	ICMP Echo Request and Reply
5	NS-001_HTTP_Traffic.png	HTTP traffic and readable HTML

16.3 Project Repository

The complete project files, documentation, screenshots, evidence, configuration files, and supporting materials are maintained in the cybersecurity project repository.